AI ASSISTED CODING ASSIGNMENT WEEK 11.1

Name: B Vrindha
HT No.:2303a52003
Batch:31

Question:1
Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
Sample Input Code:
class Stack:
pass
Expected Output:
• A functional stack implementation with all required methods and docstrings.
Code:

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        return len(self.items) == 0
    def size(self):
        return len(self.items)
# Example usage:
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek())  # Output: 2
print(stack.pop())   # Output: 2
print(stack.size())  # Output: 1
print(stack.is_empty())  # Output: False
stack.pop()
print(stack.is_empty())  # Output: True
print(stack.size())  # Output: 0
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\StackImplementation"
2
2
1
False
True
```

Justification:

- A stack follows the Last In First Out (LIFO) principle, which means the last element added is removed first.
- Operations like push, pop, peek, and is_empty help manage elements efficiently in a stack structure.

Question:2

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods

Code:

```python
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def front(self):
        if not self.is_empty():
            return self.items[0]
        return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

# Example usage:
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.front())  # Output: 1
print(queue.dequeue())   # Output: 1
print(queue.size())  # Output: 1
print(queue.is_empty())  # Output: False
queue.dequeue()
print(queue.is_empty())  # Output: True
print(queue.size())  # Output: 0
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\QueueImplementation.py"
1
1
1
False
True
0
```

Justification:

- A queue follows the First In First Out (FIFO) principle where the first element added is the first one removed.
- Methods like enqueue, dequeue, peek, and size allow proper management of elements in order.

Question:3

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def display(self):
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()
# Example usage:
linked_list = LinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\LinkedList.py"
1 2 3
```

Justification:

- A linked list stores data in nodes connected by pointers, which allows dynamic memory usage.
- It is useful for easy insertion and traversal of elements without shifting other elements.

Question:4

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, data):
        if not self.root:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert_recursive(node.right, data)
    def in_order_traversal(self):
        return self._in_order_recursive(self.root)
    def _in_order_recursive(self, node):
        result = []
        if node:
            result.extend(self._in_order_recursive(node.left))
            result.append(node.data)
            result.extend(self._in_order_recursive(node.right))
        return result
# Example usage:
bst = BST()
bst.insert(5)
bst.insert(3)
bst.insert(7)
bst.insert(2)
bst.insert(4)
bst.insert(6)
bst.insert(8)
print(bst.in_order_traversal())  # Output: [2, 3, 4, 5, 6, 7, 8]
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\BST.py"
[2, 3, 4, 5, 6, 7, 8]
```

Justification:

- A Binary Search Tree stores elements in a sorted hierarchical structure.
- It allows efficient searching, insertion, and traversal of data.

Question:5

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * self.size

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)  # Update existing key
                    return
            self.table[index].append((key, value))  # Add new key-value pair

    def search(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None  # Key not found

    def delete(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]  # Remove the key-value pair
                    return True
        return False  # Key not found
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\HashTable.py"
Alice
30
None
```

Justification:

- A hash table stores data using key-value pairs, allowing fast access to information.
- Hash functions and chaining help handle collisions and store multiple elements efficiently.

Question:6

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Code:

```
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)  # For undirected graph

    def display(self):
        for vertex, neighbors in self.adjacency_list.items():
            print(f"{vertex}: {', '.join(neighbors)}")
# Example usage:
graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.display()
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\GraphRepresentation.py"
A: B, C
B: A
C: A
```

Justification:

- A graph represents relationships between different nodes or vertices.
- Using an adjacency list helps store connections efficiently and saves memory.

Question:7

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Code:

```
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def put(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def get(self):
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        return None

    def peek(self):
        if not self.is_empty():
            return self.elements[0][1]
        return None

    def size(self):
        return len(self.elements)
# Example usage:
pq = PriorityQueue()
pq.put("task1", priority=2)
pq.put("task2", priority=1)
print(pq.peek())  # Output: task2 (highest priority)
print(pq.get())   # Output: task2
print(pq.size())  # Output: 1
print(pq.get())   # Output: task1
print(pq.is_empty())  # Output: True
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\PriorityQueue.py"
task2
task2
1
task1
True
```

Justification:

- A priority queue processes elements based on priority rather than insertion order.
- Using the heapq module ensures efficient insertion and removal of highest priority elements.

Question:8

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Code:

```
from collections import deque
class Deque:
    def __init__(self):
        self.deque = deque()

    def add_front(self, item):
        self.deque.appendleft(item)

    def add_rear(self, item):
        self.deque.append(item)

    def remove_front(self):
        if not self.is_empty():
            return self.deque.popleft()
        return None

    def remove_rear(self):
        if not self.is_empty():
            return self.deque.pop()
        return None

    def is_empty(self):
        return len(self.deque) == 0

    def size(self):
        return len(self.deque)
# Example usage:
deque_instance = Deque()
deque_instance.add_rear(1)
deque_instance.add_rear(2)
deque_instance.add_front(0)
print(deque_instance.remove_front())  # Output: 0
print(deque_instance.remove_rear())   # Output: 2
print(deque_instance.size())  # Output: 1
print(deque_instance.is_empty())  # Output: False
deque_instance.remove_rear()
print(deque_instance.is_empty())  # Output: True
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Deque.py"
0
2
1
False
True
```

Justification:

- A deque allows insertion and deletion from both front and rear ends.
- The collections.deque module provides fast and efficient double-ended operations.

Question:9

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings

Code:

```python
from collections import deque
from datetime import datetime, timedelta
import heapq
# Feature 1: Student Attendance Tracking - Deque
class AttendanceTracker:
    def __init__(self):
        self.entries = deque()
    def log_entry(self, student_id, timestamp=None):
        if timestamp is None:
            timestamp = datetime.now()
        self.entries.append({"student_id": student_id, "timestamp": timestamp, "action": "entry"})
    def log_exit(self, student_id, timestamp=None):
        if timestamp is None:
            timestamp = datetime.now()
        self.entries.append({"student_id": student_id, "timestamp": timestamp, "action": "exit"})
    def get_daily_log(self):
        return list(self.entries)
# Feature 2: Event Registration System - Hash Table
class EventRegistration:
    def __init__(self):
        self.participants = {}
    def register(self, event_id, student_id):
        if event_id not in self.participants:
            self.participants[event_id] = set()
        self.participants[event_id].add(student_id)
    def unregister(self, event_id, student_id):
        if event_id in self.participants:
            self.participants[event_id].discard(student_id)
    def search_participant(self, event_id, student_id):
        return event_id in self.participants and student_id in self.participants[event_id]
    def get_participants(self, event_id):
        return self.participants.get(event_id, set())
```

```python
# Feature 3: Library Book Borrowing - BST
class BookNode:
    def __init__(self, book_id, title, due_date):
        self.book_id = book_id
        self.title = title
        self.due_date = due_date
        self.left = None
        self.right = None
class LibraryBST:
    def __init__(self):
        self.root = None
    def insert(self, book_id, title, due_date):
        if self.root is None:
            self.root = BookNode(book_id, title, due_date)
        else:
            self._insert_recursive(self.root, book_id, title, due_date)
    def _insert_recursive(self, node, book_id, title, due_date):
        if book_id < node.book_id:
            if node.left is None:
                node.left = BookNode(book_id, title, due_date)
            else:
                self._insert_recursive(node.left, book_id, title, due_date)
        else:
            if node.right is None:
                node.right = BookNode(book_id, title, due_date)
            else:
                self._insert_recursive(node.right, book_id, title, due_date)
    def search(self, book_id):
        return self._search_recursive(self.root, book_id)
    def _search_recursive(self, node, book_id):
        if node is None:
            return None
        if book_id == node.book_id:
            return node
        elif book_id < node.book_id:
            return self._search_recursive(node.left, book_id)
        else:
            return self._search_recursive(node.right, book_id)
# Feature 4: Bus Scheduling System - Graph
class BusSchedule:
    def __init__(self):
        self.routes = {}
    def add_route(self, bus_id, stops):
        self.routes[bus_id] = stops
    def get_route(self, bus_id):
        return self.routes.get(bus_id, [])
    def find_connection(self, start_stop, end_stop):
        for bus_id, stops in self.routes.items():
            if start_stop in stops and end_stop in stops:
                return bus_id
        return None
```

```python
# Feature 5: Cafeteria Order Queue - Queue
class CafeteriaQueue:
    def __init__(self):
        self.queue = deque()
    def add_order(self, student_id, order):
        self.queue.append({"student_id": student_id, "order": order, "timestamp": datetime.now()})
    def serve_next(self):
        if self.queue:
            return self.queue.popleft()
        return None
    def queue_size(self):
        return len(self.queue)
# Main Implementation
if __name__ == "__main__":
    library = LibraryBST()
    library.insert(5, "Python Basics", datetime.now() + timedelta(days=7))
    library.insert(3, "Data Structures", datetime.now() + timedelta(days=14))
    library.insert(7, "Algorithms", datetime.now() + timedelta(days=21))

    book = library.search(5)
    if book:
        print(f"Book Found: {book.title}, Due: {book.due_date}")
    else:
        print("Book not found.")
    attendance = AttendanceTracker()
    attendance.log_entry("student_1")
    attendance.log_exit("student_1")
    print("Attendance Log:", attendance.get_daily_log())
    event_registration = EventRegistration()
    event_registration.register("event_1", "student_1")
    event_registration.register("event_1", "student_2")
    print("Event Participants:", event_registration.get_participants("event_1"))
    bus_schedule = BusSchedule()
    bus_schedule.add_route("bus_1", ["stop_a", "stop_b", "stop_c"])
    print("Bus Route for bus_1:", bus_schedule.get_route("bus_1"))
    print("Connection from stop_a to stop_c:", bus_schedule.find_connection("stop_a", "stop_c"))
    cafeteria_queue = CafeteriaQueue()
    cafeteria_queue.add_order("student_1", "Sandwich")
    cafeteria_queue.add_order("student_2", "Salad")
    print("Next order to serve:", cafeteria_queue.serve_next())
    print("Current queue size:", cafeteria_queue.queue_size())
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\CTcds.py"
Book Found: Python Basics, Due: 2026-03-17 20:41:14.501405
Attendance Log: [{'student_id': 'student_1', 'timestamp': datetime.datetime(2026, 3, 10, 20, 41, 14, 501506), 'action': 'entry'},
{'student_id': 'student_1', 'timestamp': datetime.datetime(2026, 3, 10, 20, 41, 14, 501509), 'action': 'exit'}]
Event Participants: {'student_2', 'student_1'}
Bus Route for bus_1: ['stop_a', 'stop_b', 'stop_c']
Connection from stop_a to stop_c: bus_1
Next order to serve: {'student_id': 'student_1', 'order': 'Sandwich', 'timestamp': datetime.datetime(2026, 3, 10, 20, 41, 14, 501570)}
Current queue size: 1
```

Justification:

- Different data structures help manage campus activities like attendance, event registration, and cafeteria orders efficiently.
- Structures like Queue, Hash Table, and Graph ensure fast processing, quick search, and proper order of operations.

Question:10

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
from collections import deque, defaultdict
import heapq
from datetime import datetime
from collections import deque
"""
Smart Hospital Management System - Data Structure Selection & Implementation
This program demonstrates optimal data structure choices for hospital management features.
"""
# ============================================================================
# FEATURE 1: Patient Check-In System (QUEUE)
# ============================================================================
class PatientCheckInQueue:
    """
    Uses Queue (FIFO) for patient check-in.
    Justification: Patients should be treated in order of arrival (first-come-first-served).
    Queue ensures fairness and maintains arrival sequence efficiently in O(1) time.
    """
    def __init__(self):
        self.queue = deque()
        self.patient_counter = 0
    def check_in(self, patient_name):
        """Register patient on arrival."""
        self.patient_counter += 1
        patient = {"id": self.patient_counter, "name": patient_name, "time": datetime.now()}
        self.queue.append(patient)
        print(f"✓ {patient_name} checked in (ID: {self.patient_counter})")
    def call_next_patient(self):
        """Call next patient for treatment."""
        if self.queue:
            patient = self.queue.popleft()
            print(f"→ Calling {patient['name']} (ID: {patient['id']}) for treatment")
            return patient
        print("No patients in queue")
        return None
```

```python
# ============================================================================
# FEATURE 2: Emergency Case Handling (PRIORITY QUEUE)
# ============================================================================
class EmergencyCaseHandler:
    """
    Uses Priority Queue (Min-Heap) for emergency handling.
    Justification: Critical patients must be treated first regardless of arrival time.
    Priority queue enables O(log n) insertion/extraction while maintaining urgency order.
    """
    def __init__(self):
        self.heap = []
        self.patient_counter = 0
    def add_emergency_patient(self, name, severity):
        """
        Add emergency patient with severity level (1=critical, 5=minor).
        Lower number = higher priority.
        """
        self.patient_counter += 1
        heapq.heappush(self.heap, (severity, self.patient_counter, name))
        print(f"🚨 Emergency: {name} added (Severity: {severity}/5)")
    def treat_emergency(self):
        """Treat patient with highest priority."""
        if self.heap:
            severity, _, name = heapq.heappop(self.heap)
            print(f"→ Treating emergency patient: {name} (Severity: {severity}/5)")
            return name
        print("No emergency cases")
        return None
# ============================================================================
```

```python
# ==============================================================================
# FEATURE 3: Medical Records Storage (HASH TABLE)
# ==============================================================================
class MedicalRecordsDB:
    """
    Uses Hash Table (Dictionary) for medical records.
    Justification: Fast O(1) average-case retrieval by patient ID.
    Hash tables provide instantaneous access to patient details without searching.
    """
    def __init__(self):
        self.records = {}
    def store_record(self, patient_id, record):
        """Store patient medical record."""
        self.records[patient_id] = record
        print(f"💾 Record stored for Patient ID: {patient_id}")
    def retrieve_record(self, patient_id):
        """Retrieve patient record in O(1) time."""
        if patient_id in self.records:
            print(f"📋 Retrieved record: {self.records[patient_id]}")
            return self.records[patient_id]
        print(f"❌ No record found for Patient ID: {patient_id}")
        return None
```

```python
# ============================================================================
# FEATURE 4: Doctor Appointment Scheduling (BINARY SEARCH TREE)
# ============================================================================
class TreeNode:
    def __init__(self, time, doctor, patient):
        self.time = time
        self.doctor = doctor
        self.patient = patient
        self.left = None
        self.right = None
class AppointmentBST:
    """
    Uses Binary Search Tree for appointment scheduling.
    Justification: BST maintains appointments sorted by time enabling O(log n) insertion/search.
    Range queries and sorted traversal are efficient for viewing appointment schedules.
    """
    def __init__(self):
        self.root = None
    def schedule_appointment(self, time, doctor, patient):
        """Schedule appointment (insert into BST)."""
        if not self.root:
            self.root = TreeNode(time, doctor, patient)
        else:
            self._insert(self.root, time, doctor, patient)
        print(f"📅 Appointment scheduled: {patient} with {doctor} at {time}")
    def _insert(self, node, time, doctor, patient):
        """Helper to insert appointment maintaining BST property."""
        if time < node.time:
            if node.left is None:
                node.left = TreeNode(time, doctor, patient)
            else:
                self._insert(node.left, time, doctor, patient)
        else:
            if node.right is None:
                node.right = TreeNode(time, doctor, patient)
            else:
                self._insert(node.right, time, doctor, patient)
    def view_schedule(self, node=None, schedule=None):
        """In-order traversal to view sorted appointments."""
        if schedule is None:
            schedule = []
        if node is None:
            node = self.root
        if node:
            self.view_schedule(node.left, schedule)
            schedule.append(f"{node.time}: {node.patient} → {node.doctor}")
            self.view_schedule(node.right, schedule)
                return schedule
```

```python
# ============================================================================
# FEATURE 5: Hospital Room Navigation (GRAPH)
# ============================================================================
class HospitalGraph:
    """
    Uses Graph (Adjacency List) for ward/room connections.
    Justification: Graphs represent spatial relationships between locations.
    Enables pathfinding, shortest routes, and connectivity queries efficiently.
    """
    def __init__(self):
        self.graph = defaultdict(list)
    def connect_rooms(self, room1, room2, distance):
        """Add bidirectional connection between rooms."""
        self.graph[room1].append((room2, distance))
        self.graph[room2].append((room1, distance))
        print(f"🔗 Connected {room1} ↔ {room2} (distance: {distance}m)")
    def find_shortest_path(self, start, end):
        """BFS to find shortest path between rooms."""
        visited = set()
        queue = deque([(start, [start])])
        while queue:
            room, path = queue.popleft()
            if room == end:
                print(f"🗺️  Shortest path: {' → '.join(path)}")
                return path
            if room not in visited:
                visited.add(room)
                for neighbor, _ in self.graph[room]:
                    if neighbor not in visited:
                        queue.append((neighbor, path + [neighbor]))

        print(f"❌ No path found between {start} and {end}")
        return None
```

```python
# ============================================================================
# DEMONSTRATION
# ============================================================================
def main():
    print("=" * 70)
    print("SMART HOSPITAL MANAGEMENT SYSTEM - DATA STRUCTURES DEMO")
    print("=" * 70)
    # Feature 1: Check-In Queue
    print("\n[1] PATIENT CHECK-IN (Queue)")
    checkin = PatientCheckInQueue()
    checkin.check_in("John Doe")
    checkin.check_in("Jane Smith")
    checkin.call_next_patient()
    # Feature 2: Emergency Handler
    print("\n[2] EMERGENCY CASES (Priority Queue)")
    emergency = EmergencyCaseHandler()
    emergency.add_emergency_patient("Alice", 1)
    emergency.add_emergency_patient("Bob", 4)
    emergency.add_emergency_patient("Charlie", 2)
    emergency.treat_emergency()
    # Feature 3: Medical Records
    print("\n[3] MEDICAL RECORDS (Hash Table)")
    records = MedicalRecordsDB()
    records.store_record(101, {"name": "John", "blood": "O+", "allergies": "Penicillin"})
    records.retrieve_record(101)
    records.retrieve_record(999)
    # Feature 4: Appointment Scheduling
    print("\n[4] APPOINTMENT SCHEDULING (BST)")
    appointments = AppointmentBST()
    appointments.schedule_appointment("14:30", "Dr. Smith", "John")
    appointments.schedule_appointment("10:00", "Dr. Brown", "Alice")
    appointments.schedule_appointment("16:45", "Dr. Smith", "Bob")
    schedule = appointments.view_schedule()
    print("📅 Today's Schedule (sorted by time):")
    for entry in schedule:
        print(f"   {entry}")
    # Feature 5: Room Navigation
    print("\n[5] HOSPITAL NAVIGATION (Graph)")
    hospital = HospitalGraph()
    hospital.connect_rooms("ICU", "Ward-A", 50)
    hospital.connect_rooms("Ward-A", "Surgery", 30)
    hospital.connect_rooms("ICU", "Surgery", 80)
    hospital.find_shortest_path("ICU", "Surgery")
    print("\n" + "=" * 70)
    print("✓ Demo Complete")
    print("=" * 70)
if __name__ == "__main__":
    main()
```

Output:

```
======================================================================
SMART HOSPITAL MANAGEMENT SYSTEM - DATA STRUCTURES DEMO
======================================================================

[1] PATIENT CHECK-IN (Queue)
John Doe checked in
Jane Smith checked in
Next patient: John Doe

[2] EMERGENCY CASES (Priority Queue)
Alice added with priority 1
Bob added with priority 4
Charlie added with priority 2
Treating: Alice

[3] MEDICAL RECORDS (Hash Table)
Record stored for 101
Record: {'name': 'John', 'blood': 'O+', 'allergies': 'Penicillin'}
Record not found

[4] APPOINTMENT SCHEDULING (BST)
Today's Schedule (sorted by time):
('10:00', 'Dr. Brown', 'Alice')
('14:30', 'Dr. Smith', 'John')
('16:45', 'Dr. Smith', 'Bob')

[5] HOSPITAL NAVIGATION (Graph)
Shortest distance: 80

======================================================================
Demo Complete
======================================================================
```

Justification:

- Data structures help organize patient records, appointments, and emergency handling efficiently.
- Using Queue, Priority Queue, Hash Table, and Graph ensures faster treatment, quick record access, and better hospital navigation.

Question:11

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
from collections import deque, defaultdict
import heapq
from typing import Dict, List, Tuple
from collections import deque

"""
Smart City Traffic Control System
Implements multiple data structures for traffic management features.
"""


# ============================================================================
# FEATURE 1: TRAFFIC SIGNAL QUEUE (Using Queue - FIFO)
# ============================================================================
class TrafficSignalQueue:
    """
    Queue for vehicles waiting at traffic signals.
    Justification: FIFO ensures fair order - first vehicle arrives, first leaves.
    Handles normal traffic sequentially without priority consideration.
    """
    def __init__(self):
        self.queue = deque()

    def add_vehicle(self, vehicle_id: str) -> None:
        """Add vehicle to queue."""
        self.queue.append(vehicle_id)

    def process_vehicle(self) -> str:
        """Remove and return next vehicle."""
        return self.queue.popleft() if self.queue else None

    def is_empty(self) -> bool:
        """Check if queue is empty."""
        return len(self.queue) == 0

    def queue_length(self) -> int:
        """Return current queue length."""
        return len(self.queue)
```

```python
# ============================================================================
# FEATURE 2: EMERGENCY VEHICLE PRIORITY (Using Priority Queue)
# ============================================================================
class EmergencyVehiclePriority:
    """
    Priority queue for vehicles at intersections with emergency handling.
    Justification: Priority Queue ensures ambulances/fire trucks are processed first.
    Assigns priority levels (0=emergency, 1=normal) for efficient dispatch.
    """
    def __init__(self):
        self.priority_queue = []

    def add_vehicle(self, vehicle_id: str, vehicle_type: str) -> None:
        """Add vehicle with priority based on type."""
        priority = 0 if vehicle_type in ['ambulance', 'fire_truck'] else 1
        heapq.heappush(self.priority_queue, (priority, vehicle_id))

    def get_next_vehicle(self) -> str:
        """Get highest priority vehicle."""
        if self.priority_queue:
            return heapq.heappop(self.priority_queue)[1]
        return None

    def queue_status(self) -> List[str]:
        """Return all vehicles in priority order."""
        return [v[1] for v in sorted(self.priority_queue)]
```

```python
# ============================================================================
# FEATURE 3: VEHICLE REGISTRATION LOOKUP (Using Hash Table)
# ============================================================================
class VehicleRegistrationLookup:
    """
    Hash table for instant vehicle registration lookup.
    Justification: Hash tables provide O(1) average lookup time.
    Perfect for rapid access to vehicle details like owner, model, and status.
    """
    def __init__(self):
        self.registry: Dict[str, Dict] = {}

    def register_vehicle(self, vehicle_id: str, owner: str, model: str, year: int) -> None:
        """Register a new vehicle."""
        self.registry[vehicle_id] = {
            'owner': owner,
            'model': model,
            'year': year,
            'status': 'active'
        }

    def lookup_vehicle(self, vehicle_id: str) -> Dict:
        """Lookup vehicle details."""
        return self.registry.get(vehicle_id, None)

    def update_status(self, vehicle_id: str, status: str) -> bool:
        """Update vehicle status."""
        if vehicle_id in self.registry:
            self.registry[vehicle_id]['status'] = status
            return True
        return False
```

```python
# ============================================================================
# FEATURE 4: ROAD NETWORK MAPPING (Using Graph)
# ============================================================================
class RoadNetworkMapping:
    """
    Graph representation of city road network.
    Justification: Graphs model intersections as nodes and roads as edges.
    Enables pathfinding, connectivity analysis, and network traversal.
    """
    def __init__(self):
        self.graph: Dict[str, List[Tuple[str, int]]] = defaultdict(list)

    def add_road(self, intersection1: str, intersection2: str, distance: int) -> None:
        """Add bidirectional road between intersections."""
        self.graph[intersection1].append((intersection2, distance))
        self.graph[intersection2].append((intersection1, distance))

    def get_neighbors(self, intersection: str) -> List[Tuple[str, int]]:
        """Get connected intersections."""
        return self.graph.get(intersection, [])

    def shortest_path(self, start: str, end: str) -> List[str]:
        """Find shortest path using BFS."""
        queue = deque([(start, [start])])
        visited = {start}

        while queue:
            node, path = queue.popleft()
            if node == end:
                return path

            for neighbor, _ in self.graph[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append((neighbor, path + [neighbor]))
        return []
```

```python
# ============================================================================
# FEATURE 5: PARKING SLOT AVAILABILITY (Using Hash Table)
# ============================================================================
class ParkingSlotAvailability:
    """
    Hash table for tracking parking slot status.
    Justification: Hash tables enable O(1) lookups for slot availability.
    Tracks individual slots with real-time availability updates.
    """
    def __init__(self, total_slots: int):
        self.slots: Dict[int, bool] = {i: True for i in range(total_slots)}  # True = available

    def occupy_slot(self, slot_id: int) -> bool:
        """Occupy a parking slot."""
        if slot_id in self.slots and self.slots[slot_id]:
            self.slots[slot_id] = False
            return True
        return False

    def release_slot(self, slot_id: int) -> bool:
        """Release a parking slot."""
        if slot_id in self.slots and not self.slots[slot_id]:
            self.slots[slot_id] = True
            return True
        return False

    def available_slots(self) -> int:
        """Count available slots."""
        return sum(1 for available in self.slots.values() if available)

    def get_first_available(self) -> int:
        """Get first available slot."""
        for slot_id, available in self.slots.items():
            if available:
                return slot_id
        return -1
```

```python
# ============================================================================
# DEMONSTRATION & TESTING
# ============================================================================
if __name__ == "__main__":
    print("=" * 70)
    print("SMART CITY TRAFFIC CONTROL SYSTEM - FEATURE DEMONSTRATION")
    print("=" * 70)
    # Test Traffic Signal Queue
    print("\n1. TRAFFIC SIGNAL QUEUE (Queue)")
    signal_queue = TrafficSignalQueue()
    for vehicle in ['CAR001', 'CAR002', 'CAR003']:
        signal_queue.add_vehicle(vehicle)
    print(f"   Queue length: {signal_queue.queue_length()}")
    print(f"   Processing: {signal_queue.process_vehicle()}")
    print(f"   Remaining: {signal_queue.queue_length()}")
    # Test Emergency Vehicle Priority
    print("\n2. EMERGENCY VEHICLE PRIORITY (Priority Queue)")
    emergency = EmergencyVehiclePriority()
    emergency.add_vehicle('CAR001', 'normal')
    emergency.add_vehicle('AMB001', 'ambulance')
    emergency.add_vehicle('CAR002', 'normal')
    emergency.add_vehicle('FIRE001', 'fire_truck')
    print(f"   Processing order: {emergency.queue_status()}")
    # Test Vehicle Registration
    print("\n3. VEHICLE REGISTRATION LOOKUP (Hash Table)")
    registry = VehicleRegistrationLookup()
    registry.register_vehicle('CAR001', 'John Doe', 'Toyota', 2020)
    print(f"   Lookup CAR001: {registry.lookup_vehicle('CAR001')}")
    # Test Road Network
    print("\n4. ROAD NETWORK MAPPING (Graph)")
    network = RoadNetworkMapping()
    network.add_road('A', 'B', 5)
    network.add_road('B', 'C', 3)
    network.add_road('A', 'C', 10)
    shortest = network.shortest_path('A', 'C')
    print(f"   Shortest path A→C: {' → '.join(shortest)}")
    # Test Parking Slots
    print("\n5. PARKING SLOT AVAILABILITY (Hash Table)")
    parking = ParkingSlotAvailability(5)
    print(f"   Available slots: {parking.available_slots()}")
    parking.occupy_slot(0)
    parking.occupy_slot(1)
    print(f"   After 2 occupancies: {parking.available_slots()}")
    print(f"   First available: {parking.get_first_available()}")
    print("\n" + "=" * 70)
    print("FEATURE → DATA STRUCTURE → JUSTIFICATION")
    print("=" * 70)
    print(""""
1. Traffic Signal Queue → Queue
   FIFO ensures fair, sequential vehicle processing.
2. Emergency Vehicle Priority → Priority Queue
   Prioritizes ambulances/fire trucks over regular vehicles.
3. Vehicle Registration → Hash Table
   O(1) lookup for instant vehicle detail retrieval.
4. Road Network Mapping → Graph
   Represents intersections and roads for pathfinding.
5. Parking Slot Availability → Hash Table
   O(1) availability checks and real-time updates.
    """)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\SmartCityTraffic.py"
======================================================================
SMART CITY TRAFFIC CONTROL SYSTEM - FEATURE DEMONSTRATION
======================================================================

1. TRAFFIC SIGNAL QUEUE (Queue)
   Queue length: 3
   Processing: CAR001
   Remaining: 2

2. EMERGENCY VEHICLE PRIORITY (Priority Queue)
   Processing order: ['AMB001', 'FIRE001', 'CAR001', 'CAR002']

3. VEHICLE REGISTRATION LOOKUP (Hash Table)
   Lookup CAR001: {'owner': 'John Doe', 'model': 'Toyota', 'year': 2020, 'status': 'active'}

4. ROAD NETWORK MAPPING (Graph)
   Shortest path A→C: A → C

5. PARKING SLOT AVAILABILITY (Hash Table)
   Available slots: 5
   After 2 occupancies: 3
   First available: 2

======================================================================
FEATURE → DATA STRUCTURE → JUSTIFICATION
======================================================================

1. Traffic Signal Queue → Queue
   FIFO ensures fair, sequential vehicle processing.

2. Emergency Vehicle Priority → Priority Queue
   Prioritizes ambulances/fire trucks over regular vehicles.

3. Vehicle Registration → Hash Table
   O(1) lookup for instant vehicle detail retrieval.

4. Road Network Mapping → Graph
   Represents intersections and roads for pathfinding.

5. Parking Slot Availability → Hash Table
   O(1) availability checks and real-time updates.
```

Justification:

- Data structures help manage vehicle flow, emergency vehicle priority, and parking systems efficiently.
- Structures like Queue, Priority Queue, Hash Table, and Graph allow smooth traffic management and quick data retrieval.

Question:12

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
from collections import deque, defaultdict
import heapq
from typing import Dict, List, Tuple, Set

"""
Smart E-Commerce Platform - Data Structure Implementation
Implements all 6 features with optimized data structures
"""


# ============================================================================
# FEATURE 1: Shopping Cart Management → Deque
# ============================================================================
class ShoppingCart:
    """
    Uses Deque for O(1) add/remove at both ends.
    Justification: Deque allows efficient addition and removal of products
    from both front and back, supporting undo/redo operations naturally.
    """
    def __init__(self):
        self.cart = deque()
        self.product_count = defaultdict(int)

    def add_product(self, product_id: str, quantity: int = 1):
        """Add product to cart - O(1)"""
        self.cart.append({'product_id': product_id, 'quantity': quantity})
        self.product_count[product_id] += quantity

    def remove_product(self, product_id: str):
        """Remove product from cart - O(1)"""
        self.cart = deque([item for item in self.cart if item['product_id'] != product_id])
        del self.product_count[product_id]

    def undo_last_action(self):
        """Undo last addition - O(1)"""
        if self.cart:
            removed = self.cart.pop()
            self.product_count[removed['product_id']] -= removed['quantity']

    def view_cart(self) -> List:
        """View all items in cart"""
        return list(self.cart)
```

```python
# ==============================================================================
# FEATURE 2: Order Processing System → Queue
# ==============================================================================
class OrderProcessingSystem:
    """
    Uses Queue (FIFO) for processing orders in order received.
    Justification: Queue ensures FIFO processing; orders are processed
    in the exact sequence they were placed, maintaining fairness.
    """
    def __init__(self):
        self.orders = deque()
        self.processed_orders = []

    def place_order(self, order_id: str, customer: str, items: List):
        """Add order to queue - O(1)"""
        self.orders.append({'order_id': order_id, 'customer': customer, 'items': items})

    def process_order(self) -> Dict:
        """Process next order in queue - O(1)"""
        if self.orders:
            order = self.orders.popleft()
            self.processed_orders.append(order)
            return order
        return None

    def get_pending_orders_count(self) -> int:
        """Get count of pending orders - O(1)"""
        return len(self.orders)
```

```python
# ==============================================================================
# FEATURE 3: Top-Selling Products Tracker → Priority Queue (Max Heap)
# ==============================================================================
class TopSellingTracker:
    """
    Uses Max Heap (Priority Queue) for top products ranking.
    Justification: Priority Queue allows instant access to top-selling products
    in O(log n) time and efficient updates as sales count changes.
    """
    def __init__(self):
        self.sales_count = defaultdict(int)
        self.heap = []

    def record_sale(self, product_id: str):
        """Record a sale for product - O(log n)"""
        self.sales_count[product_id] += 1
        heapq.heappush(self.heap, (-self.sales_count[product_id], product_id))

    def get_top_n_products(self, n: int = 5) -> List[Tuple]:
        """Get top N selling products - O(n log n)"""
        temp_heap = []
        results = []

        while self.heap and len(results) < n:
            count, product_id = heapq.heappop(self.heap)
            if -count == self.sales_count[product_id]:  # Valid entry
                results.append((product_id, -count))
            temp_heap.append((count, product_id))

        for item in temp_heap:
            heapq.heappush(self.heap, item)

        return results
```

```python
# ==============================================================================
# FEATURE 4: Product Search Engine → Hash Table
# ==============================================================================
class ProductSearchEngine:
    """
    Uses Hash Table for O(1) product lookup by ID.
    Justification: Hash tables provide constant-time lookup, making
    them ideal for fast product retrieval by ID.
    """
    def __init__(self):
        self.products = {}

    def add_product(self, product_id: str, name: str, price: float, stock: int):
        """Add product to index - O(1)"""
        self.products[product_id] = {
            'name': name,
            'price': price,
            'stock': stock
        }

    def search_by_id(self, product_id: str) -> Dict:
        """Search product by ID - O(1)"""
        return self.products.get(product_id, None)

    def update_stock(self, product_id: str, quantity: int):
        """Update stock - O(1)"""
        if product_id in self.products:
            self.products[product_id]['stock'] = quantity

    def get_all_products(self) -> List:
        """Get all products"""
        return list(self.products.values())
```

```python
# ==========================================================================
# FEATURE 5: Delivery Route Planning → Graph
# ==========================================================================
class DeliveryRoutePlanner:
    """
    Uses Graph for modeling warehouse and delivery locations.
    Justification: Graphs naturally represent network of warehouses
    and delivery points with distances/connections as edges.
    """
    def __init__(self):
        self.graph = defaultdict(list)

    def add_location(self, location_a: str, location_b: str, distance: float):
        """Add bidirectional connection - O(1)"""
        self.graph[location_a].append((location_b, distance))
        self.graph[location_b].append((location_a, distance))

    def find_shortest_path(self, start: str, end: str) -> Tuple[List, float]:
        """Find shortest path using Dijkstra - O((V + E) log V)"""
        distances = {location: float('inf') for location in self.graph}
        distances[start] = 0
        parent = {location: None for location in self.graph}
        visited = set()
        heap = [(0, start)]

        while heap:
            current_dist, current = heapq.heappop(heap)

            if current in visited:
                continue
            visited.add(current)

            if current == end:
                break

            for neighbor, weight in self.graph[current]:
                new_dist = current_dist + weight
                if new_dist < distances[neighbor]:
                    distances[neighbor] = new_dist
                    parent[neighbor] = current
                    heapq.heappush(heap, (new_dist, neighbor))

        # Reconstruct path
        path = []
        current = end
        while current is not None:
            path.append(current)
            current = parent[current]
        path.reverse()

        return path, distances[end]

    def get_all_connections(self, location: str) -> List:
        """Get all connections from a location - O(1)"""
        return self.graph.get(location, [])
```

```python
# ============================================================================
# FEATURE 6: Inventory Management → Binary Search Tree
# ============================================================================
class InventoryBST:
    """
    Uses BST for efficient range queries and sorted product listing.
    Justification: BST allows O(log n) search and efficient range queries
    for inventory management with sorted product IDs.
    """
    class Node:
        def __init__(self, product_id: str, quantity: int):
            self.product_id = product_id
            self.quantity = quantity
            self.left = None
            self.right = None

    def __init__(self):
        self.root = None

    def insert(self, product_id: str, quantity: int):
        """Insert product into BST - O(log n)"""
        if self.root is None:
            self.root = self.Node(product_id, quantity)
        else:
            self._insert_recursive(self.root, product_id, quantity)

    def _insert_recursive(self, node, product_id: str, quantity: int):
        if product_id < node.product_id:
            if node.left is None:
                node.left = self.Node(product_id, quantity)
            else:
                self._insert_recursive(node.left, product_id, quantity)
        else:
            if node.right is None:
                node.right = self.Node(product_id, quantity)
            else:
                self._insert_recursive(node.right, product_id, quantity)

    def search(self, product_id: str) -> int:
        """Search product quantity - O(log n)"""
        return self._search_recursive(self.root, product_id)

    def _search_recursive(self, node, product_id: str):
        if node is None:
            return -1
        if product_id == node.product_id:
            return node.quantity
        elif product_id < node.product_id:
            return self._search_recursive(node.left, product_id)
        else:
            return self._search_recursive(node.right, product_id)

    def inorder_traversal(self) -> List:
        """Get sorted products - O(n)"""
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        if node:
            self._inorder_recursive(node.left, result)
            result.append((node.product_id, node.quantity))
            self._inorder_recursive(node.right, result)
```

```python
# ============================================================================
# DEMO & TESTING
# ============================================================================
if __name__ == "__main__":
    print("=" * 70)
    print("SMART E-COMMERCE PLATFORM - DATA STRUCTURE DEMONSTRATION")
    print("=" * 70)

    # Feature 1: Shopping Cart
    print("\n1. SHOPPING CART (Deque)")
    cart = ShoppingCart()
    cart.add_product("P001", 2)
    cart.add_product("P002", 1)
    print(f"Cart: {cart.view_cart()}")
    cart.undo_last_action()
    print(f"After undo: {cart.view_cart()}")

    # Feature 2: Order Processing
    print("\n2. ORDER PROCESSING (Queue)")
    orders = OrderProcessingSystem()
    orders.place_order("O1", "Alice", ["P001", "P002"])
    orders.place_order("O2", "Bob", ["P003"])
    print(f"Pending orders: {orders.get_pending_orders_count()}")
    print(f"Processed: {orders.process_order()}")

    # Feature 3: Top Sellers
    print("\n3. TOP SELLING PRODUCTS (Priority Queue)")
    tracker = TopSellingTracker()
    for _ in range(10):
        tracker.record_sale("P001")
    for _ in range(7):
        tracker.record_sale("P002")
    for _ in range(15):
        tracker.record_sale("P003")
    print(f"Top 3 products: {tracker.get_top_n_products(3)}")

    # Feature 4: Product Search
    print("\n4. PRODUCT SEARCH (Hash Table)")
    search = ProductSearchEngine()
    search.add_product("P001", "Laptop", 999.99, 5)
    search.add_product("P002", "Mouse", 29.99, 50)
    print(f"Search P001: {search.search_by_id('P001')}")

    # Feature 5: Delivery Routes
    print("\n5. DELIVERY ROUTE PLANNING (Graph)")
    planner = DeliveryRoutePlanner()
    planner.add_location("Warehouse_A", "Hub_1", 10)
    planner.add_location("Hub_1", "Customer_X", 5)
    planner.add_location("Warehouse_A", "Customer_X", 20)
    path, dist = planner.find_shortest_path("Warehouse_A", "Customer_X")
    print(f"Shortest route: {path}, Distance: {dist}")

    # Feature 6: Inventory BST
    print("\n6. INVENTORY MANAGEMENT (BST)")
    inventory = InventoryBST()
    inventory.insert("P001", 50)
    inventory.insert("P002", 30)
    inventory.insert("P003", 70)
    print(f"Sorted inventory: {inventory.inorder_traversal()}")
    print(f"P002 quantity: {inventory.search('P002')}")
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Smartecommerceds.py"
======================================================================
SMART E-COMMERCE PLATFORM - DATA STRUCTURE DEMONSTRATION
======================================================================

1. SHOPPING CART (Deque)
Cart: [{'product_id': 'P001', 'quantity': 2}, {'product_id': 'P002', 'quantity': 1}]
After undo: [{'product_id': 'P001', 'quantity': 2}]

2. ORDER PROCESSING (Queue)
Pending orders: 2
Processed: {'order_id': 'O1', 'customer': 'Alice', 'items': ['P001', 'P002']}

3. TOP SELLING PRODUCTS (Priority Queue)
Top 3 products: [('P003', 15), ('P001', 10), ('P002', 7)]

4. PRODUCT SEARCH (Hash Table)
Search P001: {'name': 'Laptop', 'price': 999.99, 'stock': 5}

5. DELIVERY ROUTE PLANNING (Graph)
Shortest route: ['Warehouse_A', 'Hub_1', 'Customer_X'], Distance: 15

6. INVENTORY MANAGEMENT (BST)
Sorted inventory: [('P001', 50), ('P002', 30), ('P003', 70)]
P002 quantity: 30
```

Justification:

- Data structures help manage shopping carts, orders, product searches, and delivery routes efficiently.
- Using Linked List, Queue, Priority Queue, Hash Table, and Graph improves product management and order processing.

Question:13

Task Description #13: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table

o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.
Code:

```python
from collections import deque, defaultdict
import heapq
from typing import Dict, List, Tuple

"""
Smart E-Commerce Platform - Data Structure Implementation
Task: Implementing all 5 features with appropriate data structures
"""


# ============================================================================
# FEATURE 1: Shopping Cart Management (Deque)
# ============================================================================
class ShoppingCart:
    """
    Uses Deque for O(1) add/remove at both ends.
    Justification: Deque allows efficient add/remove operations at both ends,
    enabling undo/redo functionality and quick cart item management.
    """
    def __init__(self):
        self.cart = deque()

    def add_product(self, product_id: str, quantity: int) -> None:
        """Add product to cart."""
        self.cart.append({"product_id": product_id, "quantity": quantity})

    def remove_product(self) -> None:
        """Remove last added product (LIFO behavior)."""
        if self.cart:
            self.cart.pop()

    def undo_last_action(self) -> None:
        """Remove first added product (FIFO behavior)."""
        if self.cart:
            self.cart.popleft()

    def view_cart(self) -> List:
        """Return current cart items."""
        return list(self.cart)
```

```python
# ============================================================================
# FEATURE 2: Order Processing System (Queue)
# ============================================================================
class OrderQueue:
    """
    Uses Queue (FIFO) for processing orders in sequence.
    Justification: Queue ensures FIFO order processing, guaranteeing fairness
    and maintaining order placement sequence for customer satisfaction.
    """
    def __init__(self):
        self.order_queue = deque()

    def place_order(self, order_id: str, customer: str, total: float) -> None:
        """Add order to processing queue."""
        self.order_queue.append({"order_id": order_id, "customer": customer, "total": total})

    def process_order(self) -> Dict:
        """Process next order in queue."""
        if self.order_queue:
            return self.order_queue.popleft()
        return None

    def pending_orders(self) -> int:
        """Return number of pending orders."""
        return len(self.order_queue)
```

```python
# ============================================================================
# FEATURE 3: Top-Selling Products Tracker (Max Heap / Priority Queue)
# ============================================================================
class TopSellingTracker:
    """
    Uses Priority Queue (Max Heap) for ranking products by sales.
    Justification: Priority Queue enables O(log n) insertion and O(1) access
    to top-selling products, crucial for real-time analytics.
    """
    def __init__(self):
        self.sales_count = defaultdict(int)
        self.heap = []

    def record_sale(self, product_id: str, quantity: int = 1) -> None:
        """Record product sale."""
        self.sales_count[product_id] += quantity
        # Python uses min heap, negate for max heap
        heapq.heappush(self.heap, (-self.sales_count[product_id], product_id))

    def get_top_products(self, k: int = 5) -> List[Tuple[str, int]]:
        """Get top k selling products."""
        result = []
        temp_heap = []

        for _ in range(min(k, len(self.heap))):
            if self.heap:
                neg_count, product_id = heapq.heappop(self.heap)
                actual_count = -neg_count
                if actual_count == self.sales_count[product_id]:
                    result.append((product_id, actual_count))
                temp_heap.append((neg_count, product_id))

        # Restore heap
        for item in temp_heap:
            heapq.heappush(self.heap, item)

        return result
```

```python
# ============================================================================
# FEATURE 4: Product Search Engine (Hash Table)
# ============================================================================
class ProductCatalog:
    """
    Uses Hash Table for O(1) product lookup by ID.
    Justification: Hash Table provides constant-time lookup, essential for
    fast product searches in large catalogs with millions of items.
    """
    def __init__(self):
        self.products = {}

    def add_product(self, product_id: str, name: str, price: float, stock: int) -> None:
        """Add product to catalog."""
        self.products[product_id] = {
            "name": name,
            "price": price,
            "stock": stock
        }

    def search_product(self, product_id: str) -> Dict:
        """Search product by ID - O(1) operation."""
        return self.products.get(product_id, None)

    def update_stock(self, product_id: str, quantity: int) -> bool:
        """Update product stock."""
        if product_id in self.products:
            self.products[product_id]["stock"] += quantity
            return True
        return False
```

```python
# ============================================================================
# FEATURE 5: Delivery Route Planning (Graph)
# ============================================================================
class DeliveryNetwork:
    """
    Uses Graph for warehouse and delivery location connections.
    Justification: Graph efficiently represents multiple warehouse-location
    connections and enables pathfinding algorithms for optimal route planning.
    """
    def __init__(self):
        self.graph = defaultdict(list)

    def add_route(self, from_location: str, to_location: str, distance: float) -> None:
        """Add bidirectional route between locations."""
        self.graph[from_location].append((to_location, distance))
        self.graph[to_location].append((from_location, distance))

    def find_shortest_route(self, start: str, end: str) -> List[str]:
        """Find shortest route using BFS."""
        visited = set()
        queue = deque([(start, [start])])

        while queue:
            node, path = queue.popleft()
            if node == end:
                return path

            if node not in visited:
                visited.add(node)
                for neighbor, _ in self.graph[node]:
                    if neighbor not in visited:
                        queue.append((neighbor, path + [neighbor]))

        return []

    def get_routes(self, location: str) -> List[Tuple[str, float]]:
        """Get all routes from a location."""
        return self.graph[location]
```

```python
# ============================================================================
# DEMONSTRATION & TESTING
# ============================================================================
if __name__ == "__main__":
    print("=" * 70)
    print("SMART E-COMMERCE PLATFORM - DATA STRUCTURE IMPLEMENTATION")
    print("=" * 70)

    # Feature 1: Shopping Cart
    print("\n[FEATURE 1] Shopping Cart Management (Deque)")
    cart = ShoppingCart()
    cart.add_product("P001", 2)
    cart.add_product("P002", 1)
    print(f"Cart: {cart.view_cart()}")
    cart.remove_product()
    print(f"After removal: {cart.view_cart()}")

    # Feature 2: Order Queue
    print("\n[FEATURE 2] Order Processing System (Queue)")
    order_q = OrderQueue()
    order_q.place_order("O001", "Alice", 150.0)
    order_q.place_order("O002", "Bob", 200.0)
    print(f"Processing: {order_q.process_order()}")
    print(f"Pending orders: {order_q.pending_orders()}")

    # Feature 3: Top-Selling Products
    print("\n[FEATURE 3] Top-Selling Products Tracker (Priority Queue)")
    tracker = TopSellingTracker()
    tracker.record_sale("P001", 45)
    tracker.record_sale("P002", 32)
    tracker.record_sale("P003", 78)
    print(f"Top 2 products: {tracker.get_top_products(2)}")

    # Feature 4: Product Search
    print("\n[FEATURE 4] Product Search Engine (Hash Table)")
    catalog = ProductCatalog()
    catalog.add_product("P001", "Laptop", 999.99, 10)
    catalog.add_product("P002", "Mouse", 29.99, 100)
    print(f"Search P001: {catalog.search_product('P001')}")

    # Feature 5: Delivery Routes
    print("\n[FEATURE 5] Delivery Route Planning (Graph)")
    network = DeliveryNetwork()
    network.add_route("Warehouse_A", "Location_B", 10.5)
    network.add_route("Location_B", "Location_C", 5.2)
    network.add_route("Warehouse_A", "Location_C", 20.0)
    print(f"Shortest route A→C: {network.find_shortest_route('Warehouse_A', 'Location_C')}")

    print("\n" + "=" * 70)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\SmartEcommerce.py"
======================================================================
SMART E-COMMERCE PLATFORM - DATA STRUCTURE IMPLEMENTATION
======================================================================

[FEATURE 1] Shopping Cart Management (Deque)
Cart: [{'product_id': 'P001', 'quantity': 2}, {'product_id': 'P002', 'quantity': 1}]
After removal: [{'product_id': 'P001', 'quantity': 2}]

[FEATURE 2] Order Processing System (Queue)
Processing: {'order_id': 'O001', 'customer': 'Alice', 'total': 150.0}
Pending orders: 1

[FEATURE 3] Top-Selling Products Tracker (Priority Queue)
Top 2 products: [('P003', 78), ('P001', 45)]

[FEATURE 4] Product Search Engine (Hash Table)
Search P001: {'name': 'Laptop', 'price': 999.99, 'stock': 10}

[FEATURE 5] Delivery Route Planning (Graph)
Shortest route A→C: ['Warehouse_A', 'Location_C']

======================================================================
```

Justification:

- Appropriate data structures help organize product data, orders, and delivery routes efficiently.
- They provide fast search, ranking of products, and proper order handling in the shopping system.

Question:14

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List

o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.
Code:

```python
from collections import deque, defaultdict
import heapq
from datetime import datetime

"""
Smart Banking Management System - Data Structure Challenge
Implementation of multiple banking features with optimal data structures
"""


# ============================================================================
# FEATURE 1: Customer Service Token System (Queue)
# ============================================================================

class CustomerServiceQueue:
    """
    Queue-based token system for customers served in order of arrival.
    FIFO ensures fairness and maintains arrival sequence.
    """
    def __init__(self):
        self.queue = deque()
        self.token_counter = 1000

    def issue_token(self, customer_name):
        """Issue a token to a customer."""
        token = self.token_counter
        self.token_counter += 1
        self.queue.append((token, customer_name, datetime.now()))
        print(f"Token {token} issued to {customer_name}")
        return token

    def serve_customer(self):
        """Serve the next customer in queue."""
        if self.queue:
            token, name, time = self.queue.popleft()
            wait_time = (datetime.now() - time).total_seconds()
            print(f"Serving Token {token}: {name} (waited {wait_time:.2f}s)")
            return token, name
        print("No customers in queue")
        return None, None

    def get_queue_size(self):
        """Get current queue size."""
        return len(self.queue)
```

```python
# ============================================================================
# FEATURE 2: Loan Approval Priority Handling (Priority Queue)
# ============================================================================

class LoanApprovalSystem:
    """
    Priority Queue for loan processing based on urgency and amount.
    Higher priority loans processed first (max-heap using negative values).
    """
    def __init__(self):
        self.heap = []
        self.loan_id = 5000

    def apply_for_loan(self, customer_name, amount, urgency_level):
        """
        Submit loan application.
        urgency_level: 1(low) to 5(critical)
        """
        self.loan_id += 1
        # Using negative priority for max-heap behavior
        priority = -(urgency_level * 100 + amount // 1000)
        heapq.heappush(self.heap, (priority, self.loan_id, customer_name, amount, urgency_level))
        print(f"Loan {self.loan_id} applied by {customer_name}: ${amount} (Urgency: {urgency_level})")

    def process_loan(self):
        """Process the highest priority loan."""
        if self.heap:
            _, loan_id, name, amount, urgency = heapq.heappop(self.heap)
            print(f"Processing Loan {loan_id}: {name} - ${amount} (Urgency: {urgency})")
            return loan_id, name, amount
        print("No loans in queue")
        return None, None, None

    def pending_loans_count(self):
        """Get count of pending loans."""
        return len(self.heap)
```

```python
# ============================================================================
# FEATURE 3: Account Information Lookup (Hash Table)
# ============================================================================

class AccountLookupSystem:
    """
    Hash Table (Dictionary) for O(1) fast retrieval of customer details.
    Direct access by Account Number enables instant lookups.
    """
    def __init__(self):
        self.accounts = {}

    def create_account(self, account_number, customer_name, balance, email):
        """Create a new account."""
        self.accounts[account_number] = {
            'name': customer_name,
            'balance': balance,
            'email': email,
            'created': datetime.now(),
            'status': 'active'
        }
        print(f"Account {account_number} created for {customer_name}")

    def lookup_account(self, account_number):
        """Retrieve account details in O(1) time."""
        if account_number in self.accounts:
            return self.accounts[account_number]
        print(f"Account {account_number} not found")
        return None

    def update_balance(self, account_number, new_balance):
        """Update account balance."""
        if account_number in self.accounts:
            self.accounts[account_number]['balance'] = new_balance
            print(f"Balance updated for Account {account_number}: ${new_balance}")
        else:
            print(f"Account {account_number} not found")
```

```python
# ============================================================================
# FEATURE 4: Transaction History Management (Linked List via Deque)
# ============================================================================

class TransactionHistory:
    """
    Chronological transaction records using Deque for efficient appends.
    Maintains FIFO order for transaction history review.
    """
    def __init__(self, account_number):
        self.account_number = account_number
        self.transactions = deque()

    def add_transaction(self, transaction_type, amount, description):
        """Add transaction in chronological order."""
        transaction = {
            'timestamp': datetime.now(),
            'type': transaction_type,
            'amount': amount,
            'description': description
        }
        self.transactions.append(transaction)
        print(f"Transaction recorded: {transaction_type} ${amount} - {description}")

    def get_transaction_history(self, limit=10):
        """Get last N transactions."""
        history = list(self.transactions)[-limit:]
        return history

    def display_history(self):
        """Display all transactions."""
        if not self.transactions:
            print("No transactions")
            return
        for trans in self.transactions:
            print(f"  {trans['timestamp'].strftime('%Y-%m-%d %H:%M:%S')} | "
                  f"{trans['type']:10} | ${trans['amount']:10.2f} | {trans['description']}")
```

```python
# ============================================================================
# FEATURE 5: ATM Network Connectivity (Graph)
# ============================================================================

class ATMNetwork:
    """
    Graph representing ATM connectivity across cities.
    Nodes=ATMs, Edges=network connections. Enables pathfinding and routing.
    """
    def __init__(self):
        self.graph = defaultdict(list)
        self.atm_locations = {}

    def add_atm(self, atm_id, city, latitude, longitude):
        """Add ATM node to network."""
        self.atm_locations[atm_id] = {
            'city': city,
            'lat': latitude,
            'lon': longitude
        }
        print(f"ATM {atm_id} added in {city}")

    def connect_atms(self, atm1, atm2, distance):
        """Add edge between two ATMs."""
        self.graph[atm1].append((atm2, distance))
        self.graph[atm2].append((atm1, distance))
        print(f"ATMs {atm1} and {atm2} connected ({distance}km)")

    def find_nearest_atm(self, current_atm):
        """Find nearest connected ATM."""
        if current_atm in self.graph:
            neighbors = sorted(self.graph[current_atm], key=lambda x: x[1])
            if neighbors:
                nearest = neighbors[0]
                print(f"Nearest ATM to {current_atm}: {nearest[0]} ({nearest[1]}km away)")
                return nearest
        return None
```

```python
# ----------------------------------------------------------------------------
# DEMONSTRATION
# ----------------------------------------------------------------------------

if __name__ == "__main__":
    print("=" * 70)
    print("SMART BANKING MANAGEMENT SYSTEM - DATA STRUCTURE DEMONSTRATION")
    print("=" * 70)

    # Feature 1: Customer Service Queue
    print("\n1. CUSTOMER SERVICE TOKEN SYSTEM (Queue)")
    print("-" * 70)
    token_system = CustomerServiceQueue()
    token_system.issue_token("Alice Johnson")
    token_system.issue_token("Bob Smith")
    token_system.issue_token("Carol White")
    print(f"Queue size: {token_system.get_queue_size()}")
    token_system.serve_customer()
    token_system.serve_customer()

    # Feature 2: Loan Priority System
    print("\n2. LOAN APPROVAL PRIORITY SYSTEM (Priority Queue)")
    print("-" * 70)
    loan_system = LoanApprovalSystem()
    loan_system.apply_for_loan("Alice Johnson", 50000, 2)
    loan_system.apply_for_loan("Bob Smith", 200000, 5)
    loan_system.apply_for_loan("Carol White", 75000, 3)
    print(f"Pending loans: {loan_system.pending_loans_count()}")
    loan_system.process_loan()
    loan_system.process_loan()

    # Feature 3: Account Lookup
    print("\n3. ACCOUNT INFORMATION LOOKUP (Hash Table)")
    print("-" * 70)
    lookup_system = AccountLookupSystem()
    lookup_system.create_account("ACC001", "Alice Johnson", 5000, "alice@email.com")
    lookup_system.create_account("ACC002", "Bob Smith", 15000, "bob@email.com")
    account_info = lookup_system.lookup_account("ACC001")
    print(f"Account Details: {account_info}")
    lookup_system.update_balance("ACC001", 7500)

    # Feature 4: Transaction History
    print("\n4. TRANSACTION HISTORY MANAGEMENT (Deque/Linked List)")
    print("-" * 70)
    history = TransactionHistory("ACC001")
    history.add_transaction("Deposit", 2500, "Salary transfer")
    history.add_transaction("Withdrawal", 500, "ATM cash withdrawal")
    history.add_transaction("Transfer", 1000, "Bill payment")
    print("\nTransaction History:")
    history.display_history()

    # Feature 5: ATM Network
    print("\n5. ATM NETWORK CONNECTIVITY (Graph)")
    print("-" * 70)
    atm_network = ATMNetwork()
    atm_network.add_atm("ATM001", "New York", 40.7128, -74.0060)
    atm_network.add_atm("ATM002", "Boston", 42.3601, -71.0589)
    atm_network.add_atm("ATM003", "Philadelphia", 39.9526, -75.1652)
    atm_network.connect_atms("ATM001", "ATM002", 215)
    atm_network.connect_atms("ATM001", "ATM003", 95)
    atm_network.connect_atms("ATM002", "ATM003", 305)
    atm_network.find_nearest_atm("ATM001")

    print("\n" + "=" * 70)
    print("SUMMARY TABLE: Feature → Data Structure → Justification")
    print("=" * 70)
    print("""
    Feature                          | Data Structure    | Justification
    ---------------------------------|-------------------|---------------------------
    1. Customer Service Tokens       | Queue (FIFO)      | Fair order by arrival; O(1)
    2. Loan Approval Handling        | Priority Queue    | Urgent/high-value first; O(log n)
    3. Account Information Lookup    | Hash Table        | O(1) direct access by account #
    4. Transaction History           | Deque             | Chronological FIFO; O(1) append
    5. ATM Network Connectivity      | Graph             | Connections & pathfinding
    """)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\SmartBanking.py"
======================================================================
SMART BANKING MANAGEMENT SYSTEM - DATA STRUCTURE DEMONSTRATION
======================================================================

1. CUSTOMER SERVICE TOKEN SYSTEM (Queue)
----------------------------------------------------------------------
Token 1000 issued to Alice Johnson
Token 1001 issued to Bob Smith
Token 1002 issued to Carol White
Queue size: 3
Serving Token 1000: Alice Johnson (waited 0.00s)
Serving Token 1001: Bob Smith (waited 0.00s)

2. LOAN APPROVAL PRIORITY SYSTEM (Priority Queue)
----------------------------------------------------------------------
Loan 5001 applied by Alice Johnson: $50000 (Urgency: 2)
Loan 5002 applied by Bob Smith: $200000 (Urgency: 5)
Loan 5003 applied by Carol White: $75000 (Urgency: 3)
Pending loans: 3
Processing Loan 5002: Bob Smith - $200000 (Urgency: 5)
Processing Loan 5003: Carol White - $75000 (Urgency: 3)

3. ACCOUNT INFORMATION LOOKUP (Hash Table)
----------------------------------------------------------------------
Account ACC001 created for Alice Johnson
Account ACC002 created for Bob Smith
Account Details: {'name': 'Alice Johnson', 'balance': 5000, 'email': 'alice@email.com', 'created': datetime.datetime(2026, 3, 10, 21, 28, 25, 909088), 'status': 'active'}
Balance updated for Account ACC001: $7500

4. TRANSACTION HISTORY MANAGEMENT (Deque/Linked List)
----------------------------------------------------------------------
Transaction recorded: Deposit $2500 - Salary transfer
Transaction recorded: Withdrawal $500 - ATM cash withdrawal
Transaction recorded: Transfer $1000 - Bill payment

Transaction History:
  2026-03-10 21:28:25 | Deposit    | $   2500.00 | Salary transfer
  2026-03-10 21:28:25 | Withdrawal | $    500.00 | ATM cash withdrawal
  2026-03-10 21:28:25 | Transfer   | $   1000.00 | Bill payment

5. ATM NETWORK CONNECTIVITY (Graph)
----------------------------------------------------------------------
ATM ATM001 added in New York
ATM ATM002 added in Boston
ATM ATM003 added in Philadelphia
ATMs ATM001 and ATM002 connected (215km)
ATMs ATM001 and ATM003 connected (95km)
ATMs ATM002 and ATM003 connected (305km)
Nearest ATM to ATM001: ATM003 (95km away)

======================================================================
SUMMARY TABLE: Feature → Data Structure → Justification
======================================================================

   Feature                          | Data Structure    | Justification
   ---------------------------------|-------------------|-----------------------------
   1. Customer Service Tokens       | Queue (FIFO)      | Fair order by arrival; O(1)
   2. Loan Approval Handling        | Priority Queue    | Urgent/high-value first; O(log n)
   3. Account Information Lookup    | Hash Table        | O(1) direct access by account #
   4. Transaction History           | Deque             | Chronological FIFO; O(1) append
   5. ATM Network Connectivity      | Graph             | Connections & pathfinding
```

Justification:

1. Data structures help manage customer service, account information, and transaction records efficiently.
2. Using Queue, Priority Queue, Hash Table, Linked List, and Graph improves banking operations and quick data access.

Question:15

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.
Student Task:
• Select the most appropriate data structure for each feature from the given list.
• Justify your selection in 2–3 sentences per feature.
• Implement one selected feature in Python using AI-assisted code generation.
Expected Output:
• Feature → Data Structure → Justification table.
• Functional Python program with proper documentation.
Code:

```python
from collections import deque, defaultdict
from typing import Dict, List, Tuple
import heapq
import hashlib

"""
Online Learning Management System - Data Structure Selection
This module implements various features using optimized data structures.
"""


# Feature 1: Student Login Authentication (Hash Map/Dictionary)
class StudentAuthenticator:
    """
    Data Structure: Hash Map (Dictionary)
    Justification: O(1) average-case lookup time enables instant credential validation.
    Hash maps provide constant-time access to stored user credentials, making them ideal
    for frequent authentication requests without performance degradation.
    """

    def __init__(self):
        self.credentials: Dict[str, str] = {}

    def register_student(self, username: str, password: str) -> bool:
        if username in self.credentials:
            return False
        self.credentials[username] = hashlib.sha256(password.encode()).hexdigest()
        return True

    def authenticate(self, username: str, password: str) -> bool:
        if username not in self.credentials:
            return False
        hashed = hashlib.sha256(password.encode()).hexdigest()
        return self.credentials[username] == hashed
```

```python
# Feature 2: Assignment Submission Queue (FIFO Queue)
class AssignmentSubmissionQueue:
    """
    Data Structure: Queue (Deque)
    Justification: FIFO ordering ensures fairness in processing submissions chronologically.
    Deque provides O(1) enqueue and dequeue operations, maintaining submission order
    while efficiently handling high-volume concurrent submissions.
    """

    def __init__(self):
        self.submissions: deque = deque()

    def submit_assignment(self, student_id: str, assignment_id: str, content: str) -> None:
        submission = {
            'student_id': student_id,
            'assignment_id': assignment_id,
            'content': content
        }
        self.submissions.append(submission)

    def process_next_submission(self) -> Dict:
        if self.submissions:
            return self.submissions.popleft()
        return None

    def pending_count(self) -> int:
        return len(self.submissions)
```

```python
# Feature 3: Top Performer Ranking (Min-Heap)
class TopPerformerRanking:
    """
    Data Structure: Min-Heap (Priority Queue)
    Justification: Efficiently retrieves top students in O(log n) time per insertion.
    Heap-based ranking maintains sorted order while allowing dynamic updates,
    ideal for real-time leaderboard updates without full re-sorting.
    """

    def __init__(self, top_k: int = 10):
        self.top_k = top_k
        self.heap: List[Tuple[float, str]] = []

    def add_score(self, student_name: str, score: float) -> None:
        if len(self.heap) < self.top_k:
            heapq.heappush(self.heap, (score, student_name))
        elif score > self.heap[0][0]:
            heapq.heapreplace(self.heap, (score, student_name))

    def get_top_performers(self) -> List[Tuple[str, float]]:
        return sorted([(name, score) for score, name in self.heap],
                      key=lambda x: x[1], reverse=True)
```

```python
# Feature 4: Course Prerequisite Structure (Directed Acyclic Graph)
class CoursePrerequisiteDAG:
    """
    Data Structure: Adjacency List (Graph)
    Justification: Efficiently represents complex dependency relationships using O(V+E) space.
    DAG allows topological sorting and prerequisite validation, essential for course
    sequencing and curriculum planning with arbitrary complexity levels.
    """

    def __init__(self):
        self.graph: Dict[str, List[str]] = defaultdict(list)

    def add_prerequisite(self, course: str, prerequisite: str) -> None:
        self.graph[prerequisite].append(course)

    def get_prerequisites(self, course: str) -> List[str]:
        return self.graph.get(course, [])

    def topological_sort(self) -> List[str]:
        visited = set()
        stack = []

        def dfs(node):
            visited.add(node)
            for neighbor in self.graph[node]:
                if neighbor not in visited:
                    dfs(neighbor)
            stack.append(node)

        for node in self.graph:
            if node not in visited:
                dfs(node)
        return stack[::-1]
```

```python
# Feature 5: Undo Feature in Online Editor (Stack)
class OnlineEditorWithUndo:
    """
    Data Structure: Stack (LIFO List)
    Justification: LIFO structure perfectly mirrors action reversal semantics.
    Stack provides O(1) push/pop operations, maintaining action history efficiently
    while enabling instant undo/redo functionality without traversal overhead.
    """

    def __init__(self):
        self.undo_stack: List[str] = []
        self.redo_stack: List[str] = []
        self.current_content: str = ""

    def perform_action(self, action: str) -> None:
        self.undo_stack.append(self.current_content)
        self.current_content = action
        self.redo_stack.clear()

    def undo(self) -> None:
        if self.undo_stack:
            self.redo_stack.append(self.current_content)
            self.current_content = self.undo_stack.pop()

    def redo(self) -> None:
        if self.redo_stack:
            self.undo_stack.append(self.current_content)
            self.current_content = self.redo_stack.pop()

    def get_content(self) -> str:
        return self.current_content
```

```python
# Demonstration
if __name__ == "__main__":
    print("=== Online Learning Management System ===\n")

    # Test Authentication
    auth = StudentAuthenticator()
    auth.register_student("alice", "password123")
    print("Login Test:", auth.authenticate("alice", "password123"))

    # Test Assignment Queue
    queue = AssignmentSubmissionQueue()
    queue.submit_assignment("S001", "A1", "Solution")
    print("Pending submissions:", queue.pending_count())

    # Test Rankings
    ranking = TopPerformerRanking(3)
    ranking.add_score("Alice", 95)
    ranking.add_score("Bob", 88)
    print("Top Performers:", ranking.get_top_performers())

    # Test Prerequisites
    dag = CoursePrerequisiteDAG()
    dag.add_prerequisite("Advanced Python", "Python Basics")
    print("Prerequisites for Advanced Python:", dag.get_prerequisites("Advanced Python"))

    # Test Undo
    editor = OnlineEditorWithUndo()
    editor.perform_action("Hello")
    editor.perform_action("Hello World")
    editor.undo()
    print("After undo:", editor.get_content())
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\OnlineLearning.py"
=== Online Learning Management System ===

Login Test: True
Pending submissions: 1
Top Performers: [('Alice', 95), ('Bob', 88)]
Prerequisites for Advanced Python: []
After undo: Hello
```

Justification:

- Data structures help manage login authentication, assignment submissions, rankings, and course dependencies.
- Using Hash Table, Queue, Priority Queue, Graph, and Stack ensures efficient learning platform operations.